

IA Aplicada com LLMs

Aula 06 — Introdução ao RAG

Onde paramos

Quatro notas, e **nenhuma chamada de geração**. O buscador inteiro — vetor, cosseno, chunking, medida, router e um banco onde tudo isso persiste — foi construído sem que nada escrevesse uma palavra.

E ele para nos **trechos**.

Falta transformá-los em resposta.

Roteiro

1. O pipeline, e o nome dele
2. O índice sai da memória
3. O portão — e o limiar que não funciona
4. O que este pipeline **não** confere

Objetivos

- **Montar** o pipeline de RAG e **classificá-lo** na taxonomia da Aula 05
- **Justificar** o banco de vetores pelo argumento certo: persistência
- **Distinguir** distância de similaridade
- **Explicar** por que um limiar escolhido no chute não barra nada

Parte 1

O pipeline

Quatro etapas

```
pergunta —> [ busca ] —> portão —> [ contexto + LLM ] —> resposta
              |                               |
              notas 01 a 03                  esta nota
```

O nome é **RAG** — *retrieval-augmented generation* (LEWIS et al., 2020).

É o termo que o mercado usa, e é o que a Parte 2 do trabalho cobra.

E o nome da arquitetura importa igual

Pela taxonomia da Aula 05: **prompt chaining com portão**.

o caminho está no código	não é descoberto em execução
o número de chamadas é conhecido	antes de rodar
cada etapa é testável	isoladamente

Um pipeline RAG não é um agente.

Chamá-lo de agente é o *agent washing* da Aula 04.

Parte 2

O que vem da nota 04

O índice já está pronto

A nota anterior trocou a matriz pelo banco e mediu o preço. Três coisas entram aqui sem reapresentação:

o argumento	persistência, não desempenho — nesta escala não há ganho de velocidade
o retorno	é distância , e baixo é bom
o limite	o banco devolve os k mais próximos sempre

A terceira é a que importa aqui.

Se o banco sempre responde, alguém precisa decidir quando a resposta não serve.

Parte 3

O portão

A etapa que quase ninguém escreve

```
if not trechos or trechos[0]["distancia"] > piso_distancia:  
    return {"resposta": "A busca não recuperou trecho suficientemente "  
            "próximo. Encaminhado para revisão.",  
            "trechos": trechos, "barrado_no_portao": True}
```

Se a busca não trouxe nada próximo, **a geração não roda**.

Custa zero chamadas e evita uma. É a tática zero da Aula 05 num lugar novo.

Três perguntas, e a terceira não tem resposta

'qual o teto de refeição em viagem?'	[Art. 4º §1º]	d = 0,2359
'e em viagem internacional?'	[Art. 2º §3º]	d = 0,1831
'reajuste da tabela de fretes marítimos?'	[Art. 5º §1º]	d = 0,2586

O regulamento não fala de fretes marítimos. O portão está lá, o limiar é 0,6 — e ele **deixou passar**.

Por quê: o piso, de novo

limiar 0,6 de distância	=	0,4 de cosseno
piso medido na nota 01	=	0,75 de cosseno
o limiar está MUITO abaixo do piso		

A pergunta fora de domínio marcou **0,7414** de cosseno — exatamente a faixa em que a nota 01 encontrou o campeonato de futebol.

O portão está certo. O limiar foi chutado.

O que se faz com isso

Calibrar contra dados, como a nota 01 já avisava — e o material existe: o conjunto de perguntas da nota 02, acrescido de perguntas **sem** resposta no corpus.

É o que o exercício pede.

Parte 4

O que ele não confere

Quatro perguntas que este pipeline não faz

Pergunta	Onde é respondida
a resposta usou os trechos que voltaram?	Aula 07 — fidelidade
a fonte citada existe?	Aula 07 — citação verificável
o modelo deveria ter recusado?	Aula 07 — contrato de saída
qual limiar barra o que precisa?	Aula 07 — e o exercício

O que esta aula entregou

- Um **buscador inteiro** — embeddings, cosseno, chunking, índice e medida
- O primeiro **dataset de avaliação** do curso
- Uma decisão de corte **defendida com número**
- Um **router** que classifica sem gerar texto
- E um **RAG** que responde

E o que ela não entregou

Nenhuma garantia.

O pipeline roda. Ninguém conferiu se o que ele devolve se sustenta.

A distância entre "*funciona*" e "*dá para confiar*" é a Aula 07.